PONTIFÍCIA UNIVERSIDADE CATÓLICA

Karen Joilly Araújo Gregório de Almeida

IMPLEMENTAÇAO DE SOCKETS TCP E UDP EM PYTHON

Relatório Completo de Redes Sockets

Belo Horizonte

2026

1. Introdução

A comunicação entre processos em redes de computadores é fundamental para o funcionamento de aplicações distribuídas, como navegadores web, serviços de email, streaming e jogos online. Para que essa comunicação ocorra de forma eficiente, os sistemas operacionais disponibilizam uma interface de programação conhecida como sockets, que abstrai a complexidade dos protocolos de transporte.

Este relatório tem como objetivo apresentar a implementação e análise de aplicações cliente-servidor utilizando os protocolos UDP (User Datagram Protocol) e TCP (Transmission Control Protocol) por meio de sockets em Python. Foram desenvolvidos seis programas que demonstram na prática as diferenças entre os dois protocolos, especialmente no que diz respeito à confiabilidade, orientação à conexão e capacidade de atender múltiplos clientes simultaneamente.

2. Objetivo

Implementar e analisar a comunicação cliente-servidor usando sockets UDP e TCP, documentando códigos, execuções e explicações detalhadas.

3. Diferença entre UDP e TCP

Os protocolos UDP (User Datagram Protocol) e TCP (Transmission Control Protocol) operam na camada de transporte do modelo TCP/IP, mas possuem características fundamentalmente diferentes.

3.1 UDP - User Datagram Protocol

O UDP é um protocolo não orientado à conexão, o que significa que não há estabelecimento de conexão antes da transmissão dos dados. Cada mensagem (datagrama) é enviada de forma independente.

Características:

- Não há garantia de entrega
- Não há ordenação dos pacotes
- Sem controle de congestionamento

- Baixo overhead (cabeçalho pequeno)

Aplicações típicas:

- Streaming de vídeo/áudio
- Jogos online
- DNS (Domain Name System)
- VoIP (Voice over IP)

3.2 TCP - Transmission Control Protocol

O TCP é um protocolo orientado à conexão, que estabelece um canal de comunicação confiável entre cliente e servidor antes da troca de dados (three-way handshake).

Características:

- Entrega garantida com confirmação (ACK)
- Ordenação garantida dos pacotes
- Controle de fluxo
- Controle de congestionamento
- Retransmissão de dados perdidos

Aplicações típicas:

- Navegação web (HTTP/HTTPS)
- E-mail (SMTP, IMAP, POP3)
- Transferência de arquivos (FTP)
- Acesso remoto (SSH)

3.3 Comparação UDP x TCP

Característica	UDP	TCP
Conexão	Não orientado	Orientado à conexão
Confiabilidade	Não confiável	Confiável
Velocidade	Mais rápido	Mais lento

4. UDP - Cliente e Servidor

4.1 Código do Servidor (UDPServer.py)

```
"""
UDPServer.py
Este é um exemplo de um servidor UDP simples que recebe mensagens de clientes,
converte as mensagens para maiúsculas e as envia de volta para os clientes.
Parte 2
Slide 9
Karen Joilly
"""
from socket import *

serverPort = 12000 # Numero da Porta do Servidor
serverSocket = socket(AF_INET, SOCK_DGRAM) # Cria um socket UDP
serverSocket.bind(('', serverPort)) # Associa o Socket a um endereço e porta

print('The server is ready to receive') # Informa que o servidor está pronto para
receber mensagens

while True:
message, clientAddress = serverSocket.recvfrom(2048) # Recebe a mensagem do
cliente e o endereço do cliente
```

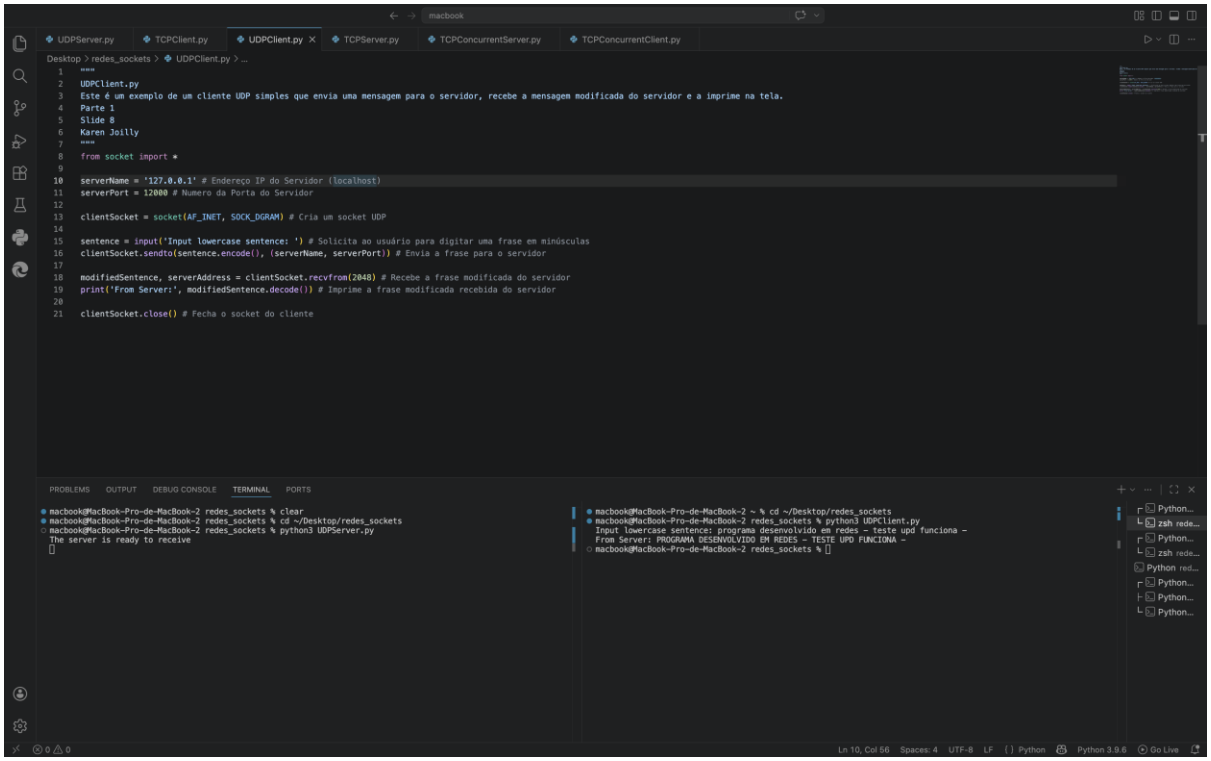
```
modifiedMessage = message.decode().upper() # Converte a mensagem para maiúsculas
serverSocket.sendto(modifiedMessage.encode(), clientAddress) # Envia a mensagem modificada de volta para o cliente
```

4.2 Código do Cliente (UDPClient.py)

```
"""
UDPClient.py
Este é um exemplo de um cliente UDP simples que envia uma mensagem para o servidor, recebe a mensagem modificada do servidor e a imprime na tela.
Parte 1
Slide 8
Karen Joilly
"""
from socket import *

serverName = '127.0.0.1' # Endereço IP do Servidor (localhost)
serverPort = 12000 # Numero da Porta do Servidor

clientSocket = socket(AF_INET, SOCK_DGRAM) # Cria um socket UDP

sentence = input('Input lowercase sentence: ') # Solicita ao usuário para digitar uma frase em minúsculas
clientSocket.sendto(sentence.encode(), (serverName, serverPort)) # Envia a frase para o servidor

modifiedSentence, serverAddress = clientSocket.recvfrom(2048) # Recebe a frase modificada do servidor
```

```
print('From Server:', modifiedSentence.decode()) # Imprime a frase modificada recebida do servidor

clientSocket.close() # Fecha o socket do cliente
```

4.3 Execução do Terminal



4.4 Explicação do Funcionamento

Servidor: O servidor UDP cria um socket utilizando a função socket(AF_INET, SOCK_DGRAM) e o associa à porta 12000 com a função bind(). Em seguida, entra em um loop infinito onde a função recvfrom() fica bloqueada aguardando a chegada de datagramas. Quando um cliente envia uma mensagem, o servidor a recebe,

decodifica os bytes com decode(), converte o texto para letras maiúsculas com upper() e devolve a resposta ao mesmo cliente utilizando sendto().

Cliente: O cliente também cria um socket UDP com socket(AF_INET, SOCK_DGRAM). Ele solicita uma frase ao usuário, codifica a string para bytes com encode() e envia essa mensagem para o servidor (endereço 127.0.0.1, porta 12000) usando sendto(). Em seguida, aguarda a resposta com recvfrom(), decodifica os bytes recebidos e exibe a mensagem na tela. Ao final, o socket é fechado com close().

Comunicação: Não há conexão prévia entre cliente e servidor, pois o UDP não realiza handshake. Cada mensagem (datagrama) é tratada de forma independente e o servidor não mantém nenhuma informação sobre o cliente após responder. Essa característica torna o UDP mais rápido que o TCP, porém menos confiável, pois não há garantia de entrega ou ordenação das mensagens.

5 TCP - Cliente e Servidor Simples

5.1 Código do Servidor (TCPServer.py)

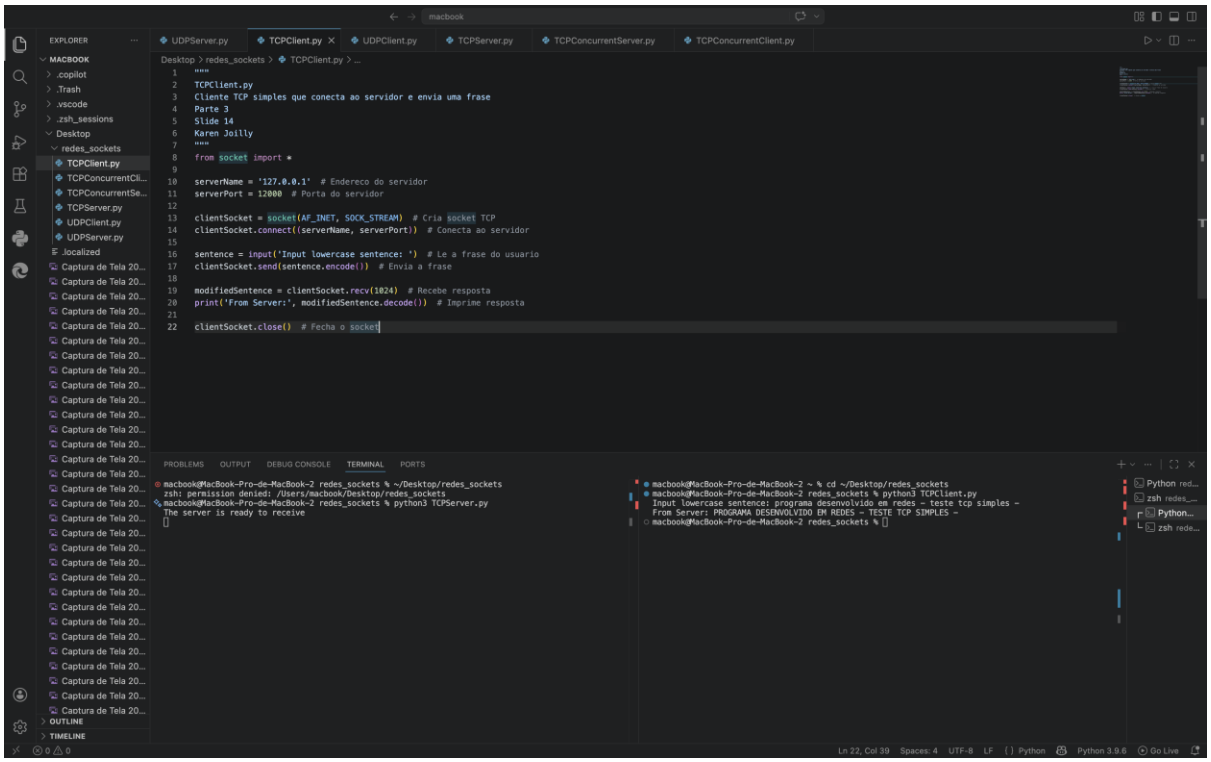
```
"""
TCPServer.py
Servidor TCP simples que aceita uma conexao, converte mensagem para
maiusculas
Parte 4
Slide 15
Karen Joilly
"""
from socket import *

serverPort = 12000 # Porta do servidor
serverSocket = socket(AF_INET, SOCK_STREAM) # Cria socket TCP
serverSocket.bind(('', serverPort)) # Associa a porta
serverSocket.listen(1) # Escuta por conexoes
```

```
print('The server is ready to receive') # Servidor pronto

while True: # Loop infinito
connectionSocket, addr = serverSocket.accept() # Aceita conexao
sentence = connectionSocket.recv(1024).decode() # Recebe dados
capitalizedSentence = sentence.upper() # Converte para maiusculas
connectionSocket.send(capitalizedSentence.encode()) # Envia resposta
connectionSocket.close() # Fecha conexao
```

5.2 Código do Cliente (TCPClient.py)

```
"""
TCPClient.py
Cliente TCP simples que conecta ao servidor e envia uma frase
Parte 3
Slide 14
Karen Joilly
"""
from socket import *

serverName = '127.0.0.1' # Endereco do servidor
serverPort = 12000 # Porta do servidor

clientSocket = socket(AF_INET, SOCK_STREAM) # Cria socket TCP
clientSocket.connect((serverName, serverPort)) # Conecta ao servidor

sentence = input('Input lowercase sentence: ') # Le a frase do usuario
clientSocket.send(sentence.encode()) # Envia a frase

modifiedSentence = clientSocket.recv(1024) # Recebe resposta
print('From Server:', modifiedSentence.decode()) # Imprime resposta

clientSocket.close() # Fecha o socket
```

5.3 Execução



5.4 Explicação do Funcionamento

O servidor TCP estabelece conexão, recebe uma mensagem, converte e envia de volta.

Servidor: O servidor TCP cria um socket e o associa à porta 12000. A função listen coloca o servidor em modo de espera por conexões. Quando um cliente se conecta, a função accept aceita a conexão e cria um novo socket específico para aquele cliente. O servidor então recebe a mensagem, converte para maiúsculas, envia de volta e fecha a conexão com aquele cliente. O servidor volta a aguardar o próximo cliente.

Cliente: O cliente cria um socket TCP e, diferentemente do UDP, precisa estabelecer uma conexão com o servidor usando a função connect. Após a conexão, envia a mensagem, aguarda a resposta, exibe na tela e fecha o socket.

Diferença para o UDP: No TCP, existe um handshake (aperto de mãos) inicial entre cliente e servidor antes do envio dos dados. Isso garante que a conexão é confiável e que os dados chegarão em ordem. O servidor TCP atende um cliente por vez, pois após aceitar uma conexão, ele só volta a aceitar outra depois que a conexão atual é fechada.

6. TCP - Servidor Concorrente

6.1 Código do Servidor (TCPConcurrentServer.py)

```
"""
TCPConcurrentServer.py
Servidor TCP concorrente que aceita multiplos clientes usando threads
Parte 5
Slide 23
Karen Joilly
"""

import socket
import _thread

HOST = '127.0.0.1' # Endereco IP do servidor
PORT = 50000 # Porta do servidor

def conectado(con, cliente):
"""Funcao executada por cada thread para atender um cliente"""
print('\nCliente conectado:', cliente)
while True:
msg = con.recv(1024) # Recebe mensagem
if not msg: # Se nao houver mensagem, cliente desconectou
break
```

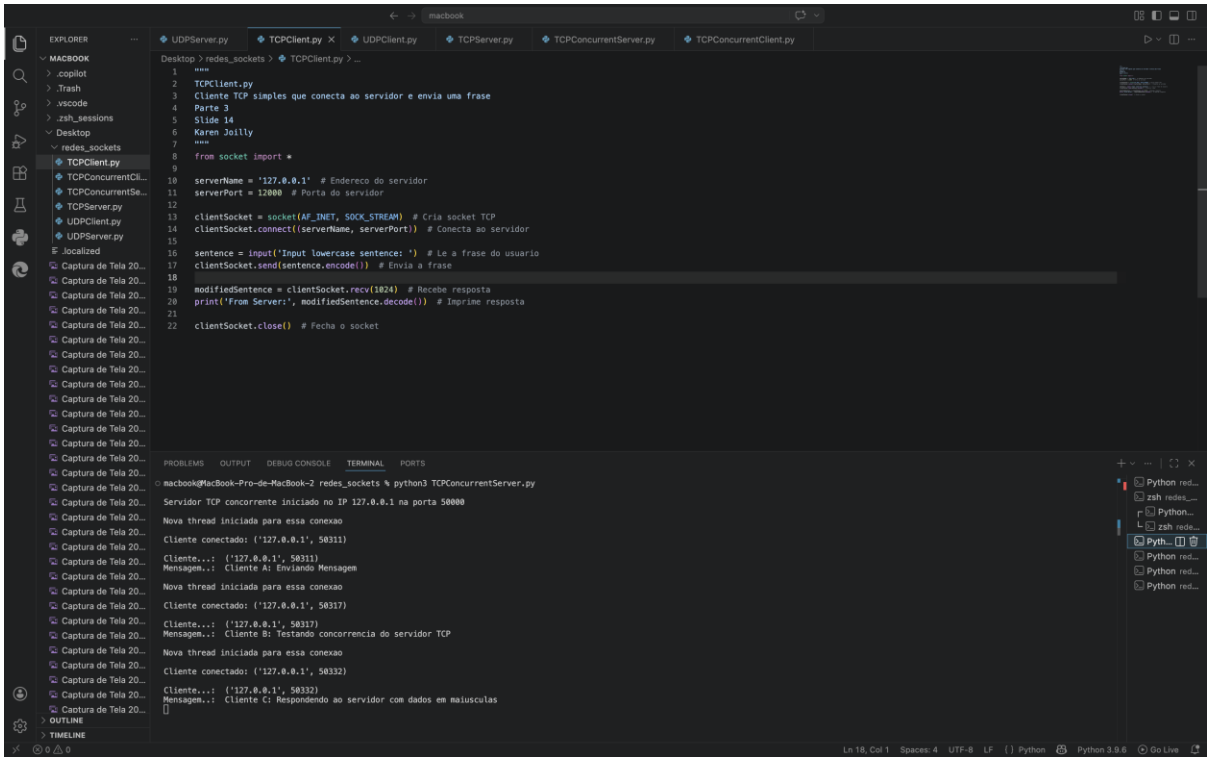
```
print('\nCliente...: ', cliente)
print('Mensagem..: ', msg.decode())
con.send(msg.upper()) # Envia resposta em maiusculas
print('\nFinalizando conexao do cliente', cliente)
con.close()
_thread.exit()

# Cria o socket principal
tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
orig = (HOST, PORT)
tcp.bind(orig) # Associa o socket a porta
tcp.listen(5) # Escuta por ate 5 conexoes pendentes
print('\nServidor TCP concorrente iniciado no IP', HOST, 'na porta', PORT)

# Loop principal do servidor
while True:
con, cliente = tcp.accept() # Aceita nova conexao
print('\nNova thread iniciada para essa conexao')
_thread.start_new_thread(conectado, (con, cliente)) # Cria thread
```

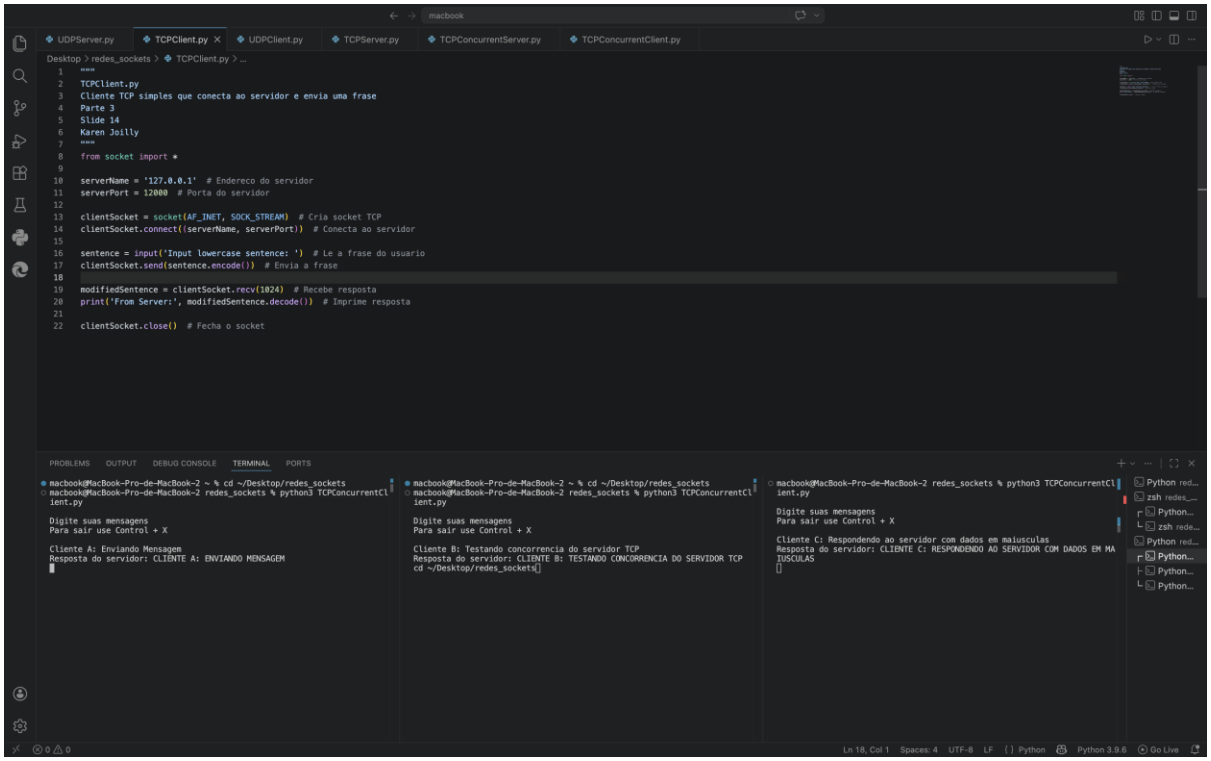
6.2 Código do Cliente (TCPConcurrentClient.py)

```
"""
TCPConcurrentClient.py
Cliente TCP para testar o servidor concorrente
Parte 6
Slide 26
Karen Joilly
"""
import socket

HOST = '127.0.0.1' # Endereco do servidor
```

```python
PORT = 50000 # Porta do servidor

# Cria e conecta o socket
tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
destino = (HOST, PORT)
tcp.connect(destino)

print('\nDigite suas mensagens')
print('Para sair use Control + X\n')

mensagem = input()
while mensagem != '\x18': # \x18 eh o caractere Control+X
tcp.send(mensagem.encode()) # Envia mensagem
resposta = tcp.recv(1024) # Recebe resposta
print('Resposta do servidor:', resposta.decode())
mensagem = input()

tcp.close() # Fecha socket
```

6.3 Execução


UDPServer.py
TCPClient.py
UDPClient.py
TCPServer.py
TCPConcurrentServer.py
TCPConcurrentClient.py
Desktop > redes_sockets > TCPClient.py > ...
"""
TCPClient.py
Cliente TCP simples que conecta ao servidor e envia uma frase
Parte 3
Slide 14
Karen Joilly
"""
from socket import *
serverName = '127.0.0.1' # Endereco do servidor
serverPort = 12000 # Porta do servidor
clientSocket = socket(AF_INET, SOCK_STREAM) # Cria socket TCP
clientSocket.connect((serverName, serverPort)) # Conecta ao servidor
sentence = input('Input lowercase sentence: ') # Le a frase do usuario
clientSocket.send(sentence.encode()) # Envia a frase
modifiedSentence = clientSocket.recv(1024) # Recebe resposta
print('From Server:', modifiedSentence.decode()) # Imprime resposta
clientSocket.close() # Fecha o socket
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
macbook@MacBook-Pro-de-MacBook-2 redes_sockets % python3 TCPConcurrentServer.py
Servidor TCP concorrente iniciado no IP 127.0.0.1 na porta 50000
Nova thread iniciada para essa conexao
Cliente conectado: ('127.0.0.1', 50311)
Cliente...: ('127.0.0.1', 50311)
Mensagem..: Cliente A: Enviando Mensagem
Nova thread iniciada para essa conexao
Cliente conectado: ('127.0.0.1', 50317)
Cliente...: ('127.0.0.1', 50317)
Mensagem..: Cliente B: Testando concorrencia do servidor TCP
Nova thread iniciada para essa conexao
Cliente conectado: ('127.0.0.1', 50332)
Cliente...: ('127.0.0.1', 50332)
Mensagem..: Cliente C: Respondendo ao servidor com dados em maiusculas


UDPServer.py
TCPClient.py
UDPClient.py
TCPServer.py
TCPConcurrentServer.py
TCPConcurrentClient.py
Desktop > redes_sockets > TCPClient.py > ...
"""
TCPClient.py
Cliente TCP simples que conecta ao servidor e envia uma frase
Parte 3
Slide 14
Karen Joilly
"""
from socket import *
serverName = '127.0.0.1' # Endereco do servidor
serverPort = 12000 # Porta do servidor
clientSocket = socket(AF_INET, SOCK_STREAM) # Cria socket TCP
clientSocket.connect((serverName, serverPort)) # Conecta ao servidor
sentence = input('Input lowercase sentence: ') # Le a frase do usuario
clientSocket.send(sentence.encode()) # Envia a frase
modifiedSentence = clientSocket.recv(1024) # Recebe resposta
print('From Server:', modifiedSentence.decode()) # Imprime resposta
clientSocket.close() # Fecha o socket
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
macbook@MacBook-Pro-de-MacBook-2 ~ % cd ~/Desktop/redes_sockets
macbook@MacBook-Pro-de-MacBook-2 redes_sockets % python3 TCPConcurrentClient.py
Digite suas mensagens
Para sair use Control + X
Cliente A: Enviando Mensagem
Resposta do servidor: CLIENTE A: ENVIANDO MENSAGEM
macbook@MacBook-Pro-de-MacBook-2 ~ % cd ~/Desktop/redes_sockets
macbook@MacBook-Pro-de-MacBook-2 redes_sockets % python3 TCPConcurrentClient.py
Digite suas mensagens
Para sair use Control + X
Cliente B: Testando concorrencia do servidor TCP
Resposta do servidor: CLIENTE B: TESTANDO CONCORRENCIA DO SERVIDOR TCP
cd ~/Desktop/redes_sockets
macbook@MacBook-Pro-de-MacBook-2 redes_sockets % python3 TCPConcurrentClient.py
Digite suas mensagens
Para sair use Control + X
Cliente C: Respondendo ao servidor com dados em maiusculas
Resposta do servidor: CLIENTE C: RESPONDENDO AO SERVIDOR COM DADOS EM MAIUSCULAS

6.4 Explicação do Funcionamento

Cria uma nova thread para cada cliente, permitindo múltiplas conexões simultâneas.

Problema do servidor TCP simples: Ele atende apenas um cliente por vez. Enquanto um cliente está conectado, os outros ficam esperando na fila.

Solução do servidor concorrente: O servidor cria uma nova thread (linha de execução) para cada cliente que se conecta. A thread principal fica apenas aceitando novas conexões. Cada thread criada é responsável por receber e responder às mensagens de um cliente específico.

Funcionamento passo a passo:

1. O servidor cria um socket e fica escutando na porta 50000.
2. Quando um cliente se conecta, o servidor aceita a conexão e imediatamente cria uma nova thread.
3. A função conectado é executada dentro dessa nova thread, cuidando da comunicação com aquele cliente.
4. Enquanto isso, a thread principal volta a aguardar o próximo cliente.
5. Cada cliente pode enviar quantas mensagens quiser, e o servidor responde convertendo para maiúsculas.
6. Quando o cliente encerra a conexão (CTRL+X), a thread é finalizada.

Vantagem: Múltiplos clientes podem conversar com o servidor ao mesmo tempo, sem que um precise esperar o outro terminar.

7. Conclusão

A implementação e execução dos seis programas desenvolvidos neste trabalho permitiram compreender na prática as principais diferenças entre os protocolos UDP e TCP. O protocolo UDP mostrou-se mais rápido por não estabelecer conexão antes do envio dos dados, porém não oferece garantia de entrega das mensagens, sendo indicado para aplicações como streaming de vídeo e jogos online. Já o TCP

demonstrou ser confiável, pois estabelece uma conexão através do handshake antes da troca de dados e garante que todas as mensagens cheguem em ordem, mas sua versão simples atende apenas um cliente por vez. Essa limitação foi resolvida com o servidor TCP concorrente, que utiliza threads para atender múltiplos clientes simultaneamente, assim como ocorre em servidores reais da internet. Todos os códigos foram testados com sucesso e o endereço do servidor foi alterado para 127.0.0.1 (localhost), conforme solicitado pelo professor. O código fonte completo está disponível no repositório GitHub: https://github.com/karenjoilly11/redes_sockets

8. Referências

Slides REDES3_ES_TCP_Sockets_Parte2.pdf. Disciplina de Redes de Computadores.

Repositório GitHub - redes_sockets. Disponível em: https://github.com/karenjoilly11/redes_sockets